Лабораторная работа 1 «Решение задач ONEMAX генетическим алгоритмом».

Реализация структуры генетического алгоритма (ГА).



Популяция состоит из списков, содержащих нули и единицы:



Наша цель – найти решение, которое бы давало максимальную сумму цифр этого списка:

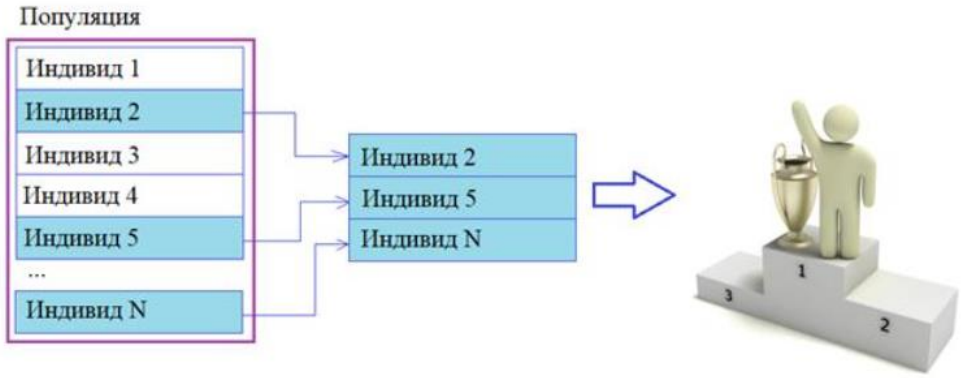
$$fitness = \sum_{i=0}^{N-1} individ[i]$$

Здесь N – длина списка. Очевидно, лучший, наиболее приспособленный индивид, должен состоять из всех единиц:

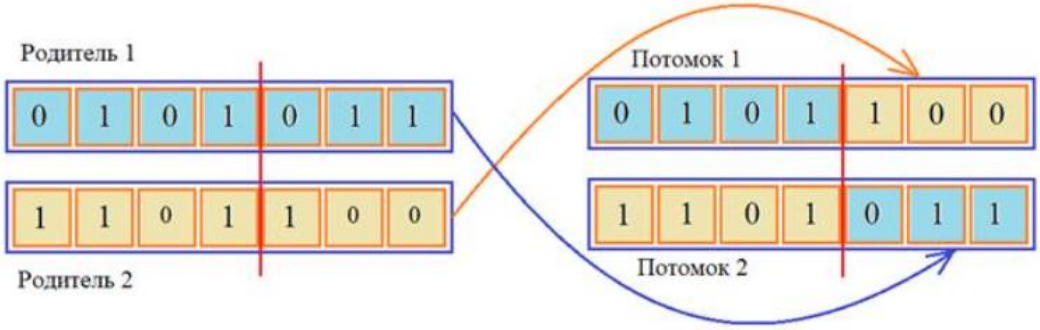


Это и есть решение данной задачи.

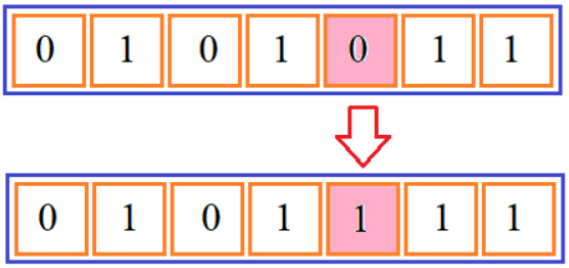
В рамках задачи OneMax мы воспользуемся идеей турнирного отбора с выборкой из трех претендентов на каждой итерации:



Будем использовать одноточечное скрещивание (одноточечный кроссинговер):

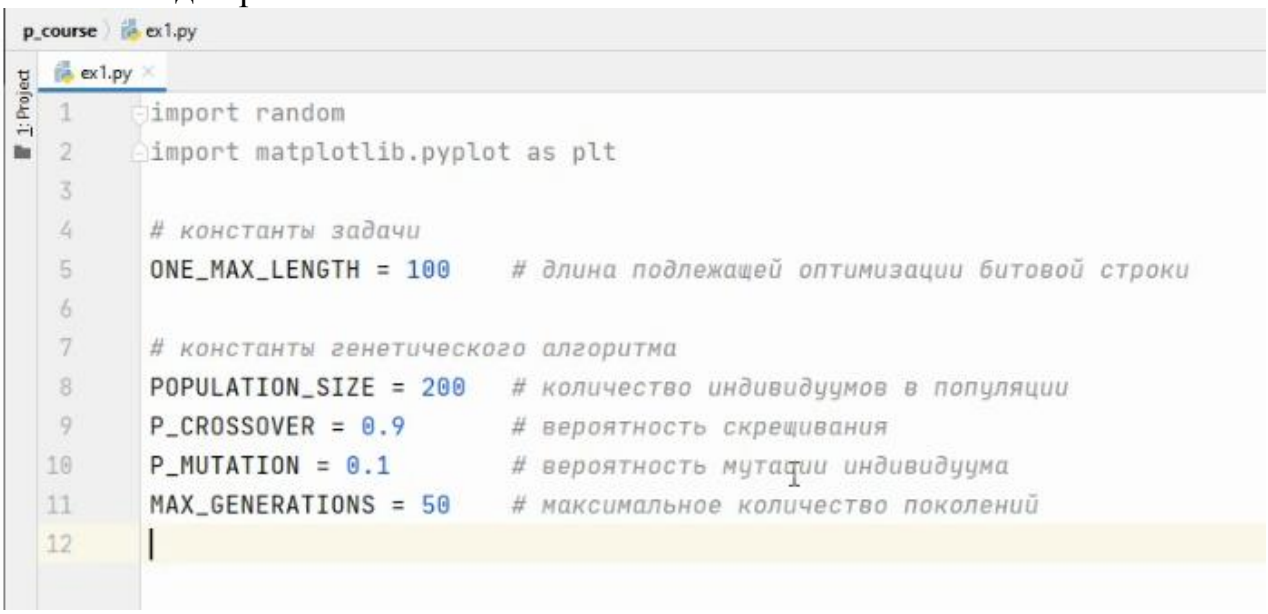


А при мутации будем выполнять инвертирование бита с некоторой очень небольшой вероятностью:

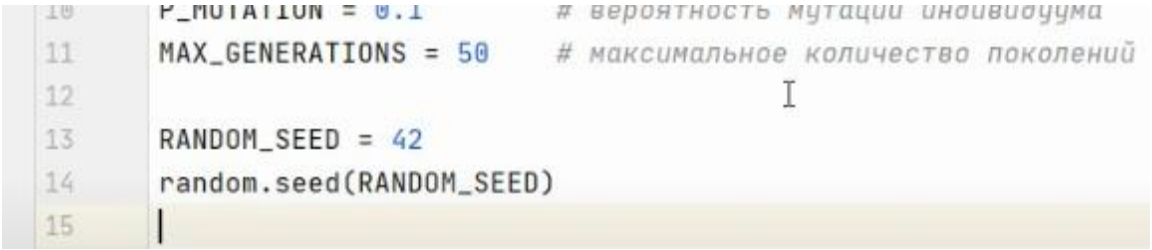


Реализация задачи OneMax на Python

Вначале подключим необходимые библиотеки и зададим глобальные константы для работы нашего ГА.



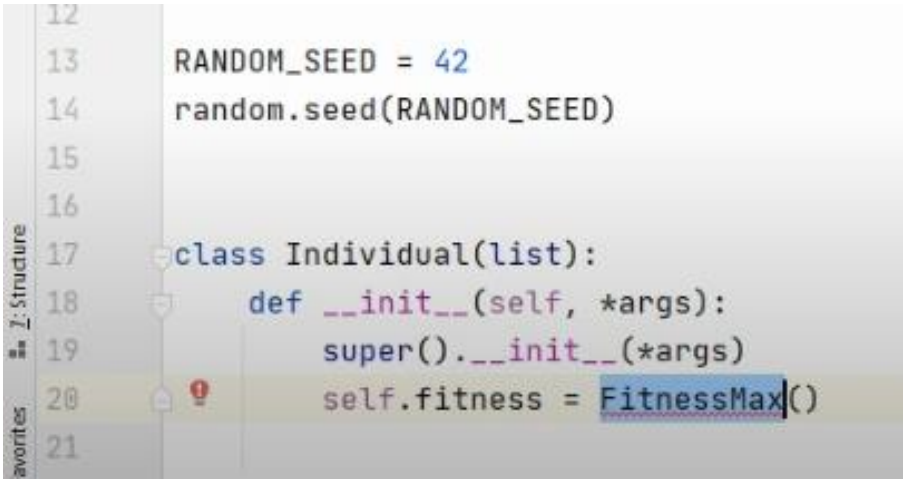
Далее, зафиксируем счетчик псевдослучайных чисел, так, чтобы и у меня и у вас он выдавал одну и ту же реализацию случайных значений:



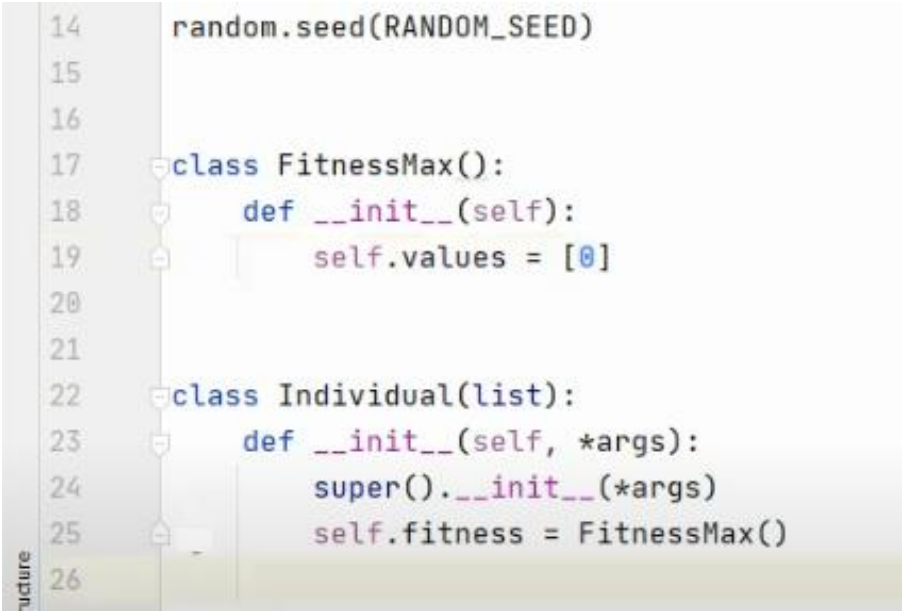
Таким образом, будет обеспечена воспроизводимость эксперимента в разное время и на разных устройствах.

Для представления каждого индивидуума в популяции, мы объявим класс с именем **Individual,** который будет наследоваться от базового класса **list** (это класс для представления списков в **Python**). То есть, хромосома особи будет представлена обычным списком и содержать целые числа **0** или **1**:

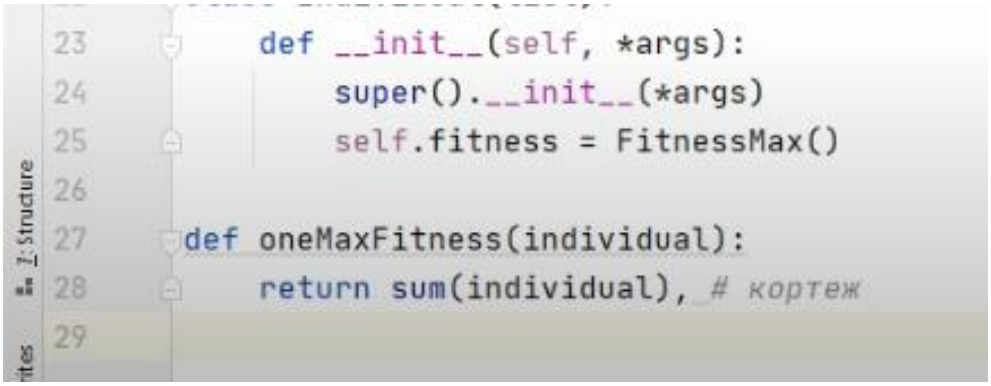
Определен конструктор класса **def_ _ init(self, *args):** в котором вызывается с начало конструктор базового класса **super()._ _ init_ _(*args**), т.е. **list,** а затем дополнительно мы определяем **fitness** которое будет связано с вычислением приспособленности данного индивида. Класс **FitnessMax** создается для обеспечения совместимости с пакетом **DEAP** (будет использоваться в следующих лабораторных работах).



Добавляем класс **FitnessMax** – идет обычный конструктор **def_ _ init _ _(self):** в котором создается свойство **values** с одном значением **0**, т.е. начальная приспособленность особи будет принимать нулевое значение.



Далее определим функцию принадлежности, которая будет вычислять приспособленность отдельной особи. В качестве аргумента будем передавать **individual** экземпляр **class Individual** и далее вызывать функцию **sum**, которая будет суммировать все значения списка **list.** (т.е. реализму $fitness = \sum_{i=0}^{N-1} individ[i]$).



```
23      def __init__(self, *args):
24          super().__init__(*args)
25          self.fitness = FitnessMax()
26
27  def oneMaxFitness(individual):
28      return sum(individual), # кортеж
29
```

Пропишем еще две вспомогательные функции.

Функции **individualCreator** будет создавать отдельного индивидуума. При создании **Individual** вызывается конструктор класса (22 строка) и ему передается список

[random.randint(0,1) for i in range(ONE_MAX_LENGTH)]

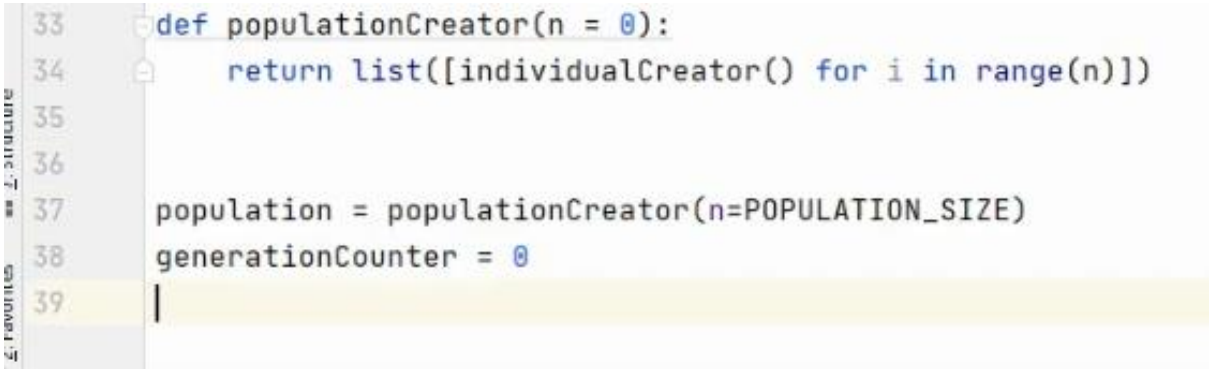
Функция **populationCrtator** будет создавать всю популяцию размера **n**. Вызывается функция **individualCreator** и формируется список из соответствующих индивидуумов **list([individualCreator() for i in range(n)**]). Т.е. популяция представляется обычным списком языка Python.



```
28      return sum(individual), # кортеж
29
30  def individualCreator():
31      return Individual([random.randint(0, 1) for i in range(ONE_MAX_LENGTH)])
32
33  def populationCreator(n = 0):
34      return list([individualCreator() for i in range(n)])
35
```

Далее вызовем функцию **populationCrtator** с размером популящии, который определен параметрами **n=POPULATION_SIZE.**

Счетчик числа поколений будет **generationCounter=0.**



```
33  def populationCreator(n = 0):
34      return list([individualCreator() for i in range(n)])
35
36
37  population = populationCreator(n=POPULATION_SIZE)
38  generationCounter = 0
39
```

Далее вычисляем текущие значения приспособленности для каждой особи в нашей сформированной популяции. Для этого выполним

fitnessValues = list(map(oneMaxFitness, population))

воспользуемся функцией **map** и для каждого индивидуума **population** вызовем функцию **oneMaxFitness** (т.е**.** эта функция, которая вычисляет приспособленность каждой отдельной особи - 28 строка).

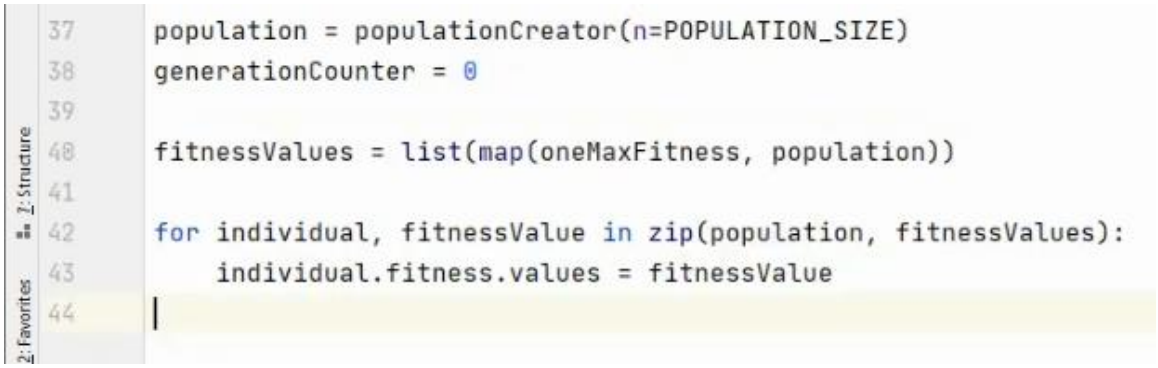
В результате получится список (**fitnessValues)** приспособленности особей в текущей популящии.

Затем в цикле

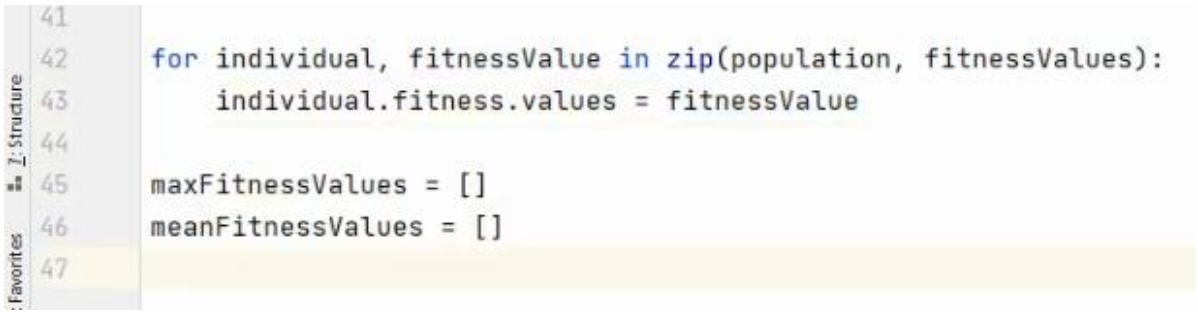
for individual, fitnessValues in zip(population, fitnessValues):

individual. fitness.values = fitnessValues

свойство **values,** которое хранится в **class FitnessMax** (17 строчка) мы присваиваем вычисленные значение функции приспособленности **fitnessValues**.

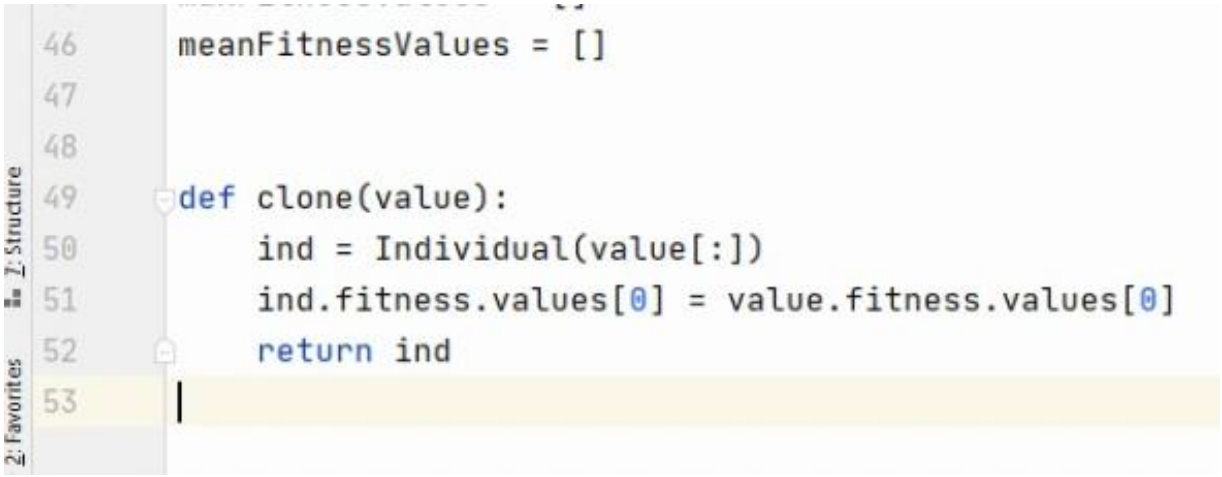


Далее определяем два списка, которые будут хранить статистику работы алгоритма. **max**F**itnessValues** будет определять максимальную приспособленность особей в текущей популяции, а **mean**F**itnessValues** среднюю приспособленность всех особей в текущей популяции.



Далее используем функции для реализации генетического алгоритма - клонирование отдельного индивидуума. Она необходима, когда после отбора мы должны будем про клонировать каждого индивидуума. Потому, что в процессе отбора может быть отобран один и тот же индивидуум дважды и у нас в популяции будет две ссылки на один и тот же список. Это недопустимо.

Каждый индивидуума должен быть представлен своим списком. Именно поэтому мы делаем клонирование. Клонирование работает очень просто. Мы создаем новый экземпляр класса (**Individual**) и передаем ему список (**value[:]**), а затем обновляем свойство (**values[0]**).



```
meanFitnessValues = []


def clone(value):
    ind = Individual(value[:])
    ind.fitness.values[0] = value.fitness.values[0]
    return ind

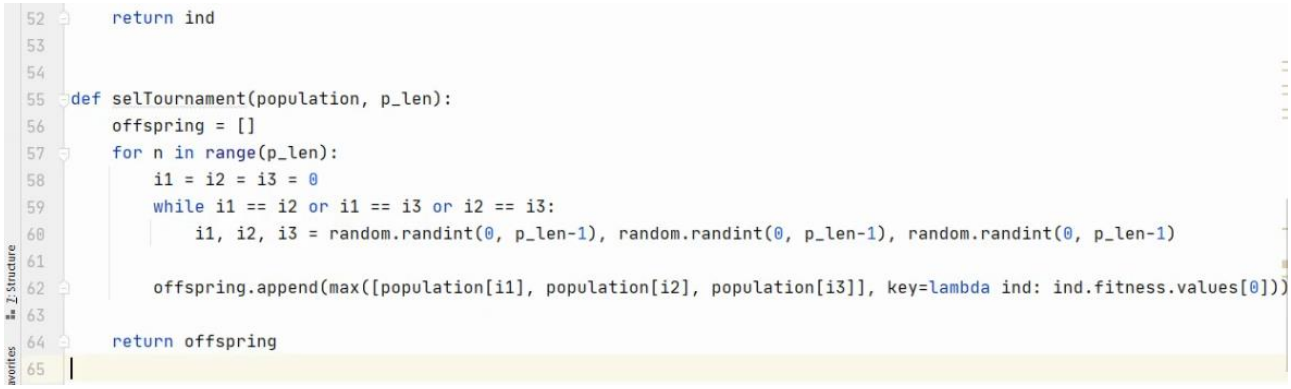
```

Далее будет функция турнирного отбора. Здесь мы передаем популяцию **population** и размеры этой популяции **p_len.**

Далее **offspring** позволяет формировать новый список из отобранных особей. Далее цикл по всей популяции **(in range(p_len)**.

Во вложенном цикле (**while i1==i2 or i1==i3 i2==i3:**) мы случайным образом отбираем трех особей, так, чтобы у них индексы **i1, i2, i3** были различные. Затем, среди этих тех особей отбираем ту (**offspring**…..), у которой максимальная приспособленность (**key=lambda**).

Когда процедура была выполнена **p_len** раз будет сформирована популяция того же размера, что была и прежде, мы возвращаем эту коллекцию (**return offspring).**



```
    return ind


def selTournament(population, p_len):
    offspring = []
    for n in range(p_len):
        i1 = i2 = i3 = 0
        while i1 == i2 or i1 == i3 or i2 == i3:
            i1, i2, i3 = random.randint(0, p_len-1), random.randint(0, p_len-1), random.randint(0, p_len-1)

        offspring.append(max([population[i1], population[i2], population[i3]], key=lambda ind: ind.fitness.values[0]))

    return offspring

```

Следующая функция одноточечного скрещивания. На ее вход мы передаем двух родителей (**child1**, **child2**) и на основе этих родителей формируем двух потомков (**child1[s:], child2[s:]). s** – это точка разреза хромосомы, определяется случайным образом, но так, чтобы границы не попадали (-3). Затем меняем части хромосом.

```
    return offspring

def cxOnePoint(child1, child2):
    s = random.randint(2, len(child1)-3)
    child1[s:], child2[s:] = child2[s:], child1[s:]

```

Последняя функция – мутация (**mutFlipBit**). Параметр **indpb=0.01** вероятность мутации отдельного гена. Сама мутация происходит в строчки

mutant[indx]=0 if mutant[indx]= = 1 else 1

т.е. происходит инверсия бита (1 меняем на 0, а 0 меняем на 1).

```
    child1[s:], child2[s:] = child2[s:], child1[s:]


def mutFlipBit(mutant, indpb=0.01):
    for indx in range(len(mutant)):
        if random.random() < indpb:
            mutant[indx] = 0 if mutant[indx] == 1 else 1

```

Перед главным циклом ГА сформируем коллекцию **fitnessValues**, которая будет состоять из значений приспособленности особей в данной популяции.

```
            mutant[indx] = 0 if mutant[indx] == 1 else 1


fitnessValues = [individual.fitness.values[0] for individual in population]

```

Главный цикл работы ГА выглядит следующим образом. В начале мы делаем цикл (**while….**)до тех пор, пока на найдем лучшее решение (**ONE_MAX_LENGTH**) либо не пройдем 50 поколений (**MAX_GENERATIONS**).

Далее подсчитываем число поколений **generationCounter += 1**. Затем отбираем лучших особей с помощью турнирного отбора (**offspring= selTournament…..**), клонируем их (**offspring= list…**) чтобы не было повторений.

Затем выполняем скрещивание (**for child1**,) причем выбираем из списка четный и нечетный элемент, т.е. не повторяющиеся пары родителей на основе

которых с вероятностью **P_CROSSOVER** формируем двух потомков **child1**, **child2**.

Если условие **if random.random() < P_CROSSOVER** сработало, то родители превращаются в потомков **cxOnePoint(child1, child2)**, а если не сработало, то родители проста остаются в популяции, с ними ничего не происходит.

Далее выполняется мутация (**for mutant**…). Вероятность мутации определяется как 1 деленная на длину хромосом (**indpb=1.0/ONE_MAX_LENGTH**), но сначала индивидуум выбирается для мутации (**if random.random() < P_MUTATION**).

```
74
75  fitnessValues = [individual.fitness.values[0] for individual in population]
76
77  while max(fitnessValues) < ONE_MAX_LENGTH and generationCounter < MAX_GENERATIONS:
78      generationCounter += 1
79      offspring = selTournament(population, len(population))
80      offspring = list(map(clone, offspring))
81
82      for child1, child2 in zip(offspring[::2], offspring[1::2]):
83          if random.random() < P_CROSSOVER:
84              cxOnePoint(child1, child2)
85
86      for mutant in offspring:
87          if random.random() < P_MUTATION:
88              mutFlipBit(mutant, indpb=1.0/ONE_MAX_LENGTH)
89
```

Далее следующей строчкой мы обновляем значение приспособленности особей новой популяции. Записываем их в свойство **values** непосредственно для каждого **individual** и обновляем список популяции **population[:]=offspring**.

Далее обновляем **fitnessValues** значение приспособленности каждой особи в популяции.

```
87          if random.random() < P_MUTATION:
88              mutFlipBit(mutant, indpb=1.0/ONE_MAX_LENGTH)
89
90      freshFitnessValues = list(map(oneMaxFitness, offspring))
91      for individual, fitnessValue in zip(offspring, freshFitnessValues):
92          individual.fitness.values = fitnessValue
93
94      population[:] = offspring
95
96      fitnessValues = [ind.fitness.values[0] for ind in population]
97
```

Далее формируем статистику. Особь с максимальной приспособленностью **maxFitness**, средняя приспособленность всех особей в популяции **meanFitness** и добавляем эти значения в списки.

Затем выводим служебную информацию (**print…**) в консоль на каждой итерации работы алгоритма, индекс лучшего индивидуума (**best_index**) и его хромосом (***population**[**best_index**],)

```
97
98  maxFitness = max(fitnessValues)
99  meanFitness = sum(fitnessValues) / len(population)
100 maxFitnessValues.append(maxFitness)
101 meanFitnessValues.append(meanFitness)
102 print(f"Поколение {generationCounter}: Макс приспособ. = {maxFitness}, Средняя приспособ.= {meanFitness}")
103
104 best_index = fitnessValues.index(max(fitnessValues))
105 print("Лучший индивидуум = ", *population[best_index], "\n")
```

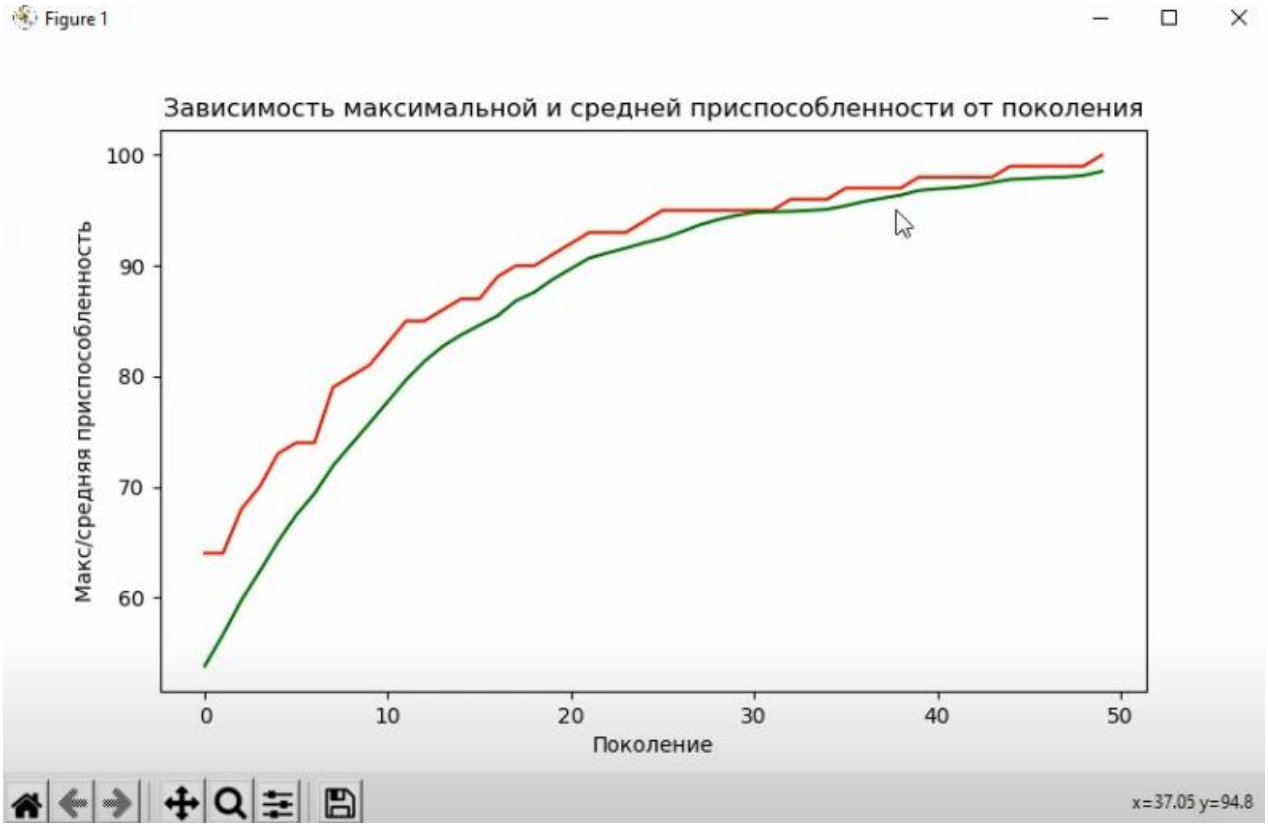
После отработки алгоритма выведем собранную статистику в виде графиков.

```
104     best_index = fitnessValues.index(max(fitnessValues))
105     print("Лучший индивидуум = ", *population[best_index], "\n")
106
107
108 plt.plot(maxFitnessValues, color='red')
109 plt.plot(meanFitnessValues, color='green')
110 plt.xlabel('Поколение')
111 plt.ylabel('Макс/средняя приспособленность')
112 plt.title('Зависимость максимальной и средней приспособленности от поколения')
113 plt.show()
114
```

Запускаем программу и смотрим на результат. Максимальна приспособленность равна 100, т.е. это лучшее решение, состоящее из всех единиц.

```
Run:  ex1
Лучший индивидуум =  1 1 1 
```

График: красная линия – это максимальная приспособленность, а зеленая линия – это средняя приспособленность. Лучший индивидуум был, достигнут в 50 поколении.



Таким образом, ГА по имитации эволюции позволил получить наиболее приспособленную особь, состоящую из всех 1, т.е. решение поставленной задачи было найдено.

Получилась довольно большая программа. Использование пакета **DEAP** для ГА позволяет значительно сократить программу.

Программа решения задачи ONEMAX генетическим алгоритмом.

ex1.py

```
import random
import matplotlib.pyplot as plt

# константы задачи
ONE_MAX_LENGTH = 100    # длина подлежащей оптимизации битовой строки

# константы генетического алгоритма
POPULATION_SIZE = 200   # количество индивидуумов в популяции
P_CROSSOVER = 0.9       # вероятность скрещивания
P_MUTATION = 0.1        # вероятность мутации индивидуума
MAX_GENERATIONS = 50    # максимальное количество поколений

RANDOM_SEED = 42
random.seed(RANDOM_SEED)


class FitnessMax():
    def __init__(self):
        self.values = [0]


class Individual(list):
    def __init__(self, *args):
        super().__init__(*args)
        self.fitness = FitnessMax()

def oneMaxFitness(individual):
    return sum(individual), # кортеж

def individualCreator():
    return Individual([random.randint(0, 1) for i in range(ONE_MAX_LENGTH)])

def populationCreator(n = 0):
    return list([individualCreator() for i in range(n)])

```

```
35
36
37  population = populationCreator(n=POPULATION_SIZE)
38  generationCounter = 0
39
40  fitnessValues = list(map(oneMaxFitness, population))
41
42  for individual, fitnessValue in zip(population, fitnessValues):
43      individual.fitness.values = fitnessValue
44
45  maxFitnessValues = []
46  meanFitnessValues = []
47
48
49  def clone(value):
50      ind = Individual(value[:])
51      ind.fitness.values[0] = value.fitness.values[0]
52      return ind
53
54
55  def selTournament(population, p_len):
56      offspring = []
57      for n in range(p_len):
58          i1 = i2 = i3 = 0
59          while i1 == i2 and i1 == i3 and i2 == i3:
60              i1, i2, i3 = random.randint(0, p_len-1), random.randint(0, p_len-1), random.randint(0, p_len-1)
61
62          offspring.append(max([population[i1], population[i2], population[i3]], key=lambda ind: ind.fitness.values[0]))
63
```

ex1.py

```
63
64      return offspring
65
66  def cxOnePoint(child1, child2):
67      s = random.randint(2, len(child1)-3)
68      child1[:s], child2[s:] = child2[:s], child1[s:]
69
70  def mutFlipBit(mutant, indpb=0.01):
71      for indx in range(len(mutant)):
72          if random.random() < indpb:
73              mutant[indx] = 0 if mutant[indx] == 1 else 1
74
75  fitnessValues = [individual.fitness.values[0] for individual in population]
76
77  while max(fitnessValues) < ONE_MAX_LENGTH and generationCounter < MAX_GENERATIONS:
78      generationCounter += 1
79      offspring = selTournament(population, len(population))
80      offspring = list(map(clone, offspring))
81
82      for child1, child2 in zip(offspring[::2], offspring[1::2]):
83          if random.random() < P_CROSSOVER:
84              cxOnePoint(child1, child2)
85
86      for mutant in offspring:
87          if random.random() < P_MUTATION:
88              mutFlipBit(mutant, indpb=1.0/ONE_MAX_LENGTH)
89
```

```
89
90      freshFitnessValues = list(map(oneMaxFitness, offspring))
91      for individual, fitnessValue in zip(offspring, freshFitnessValues):
92          individual.fitness.values = fitnessValue
93
94      population[:] = offspring
95
96      fitnessValues = [ind.fitness.values[0] for ind in population]
97
98      maxFitness = max(fitnessValues)
99      meanFitness = sum(fitnessValues) / len(population)
100     maxFitnessValues.append(maxFitness)
101     meanFitnessValues.append(meanFitness)
102     print(f"Поколение {generationCounter}: Макс приспособ. = {maxFitness}, Средняя приспособ.= {meanFitness}")
103
104     best_index = fitnessValues.index(max(fitnessValues))
105     print("Лучший индивидуум = ", *population[best_index], "\n")
106
107
108 plt.plot(maxFitnessValues, color='red')
109 plt.plot(meanFitnessValues, color='green')
110 plt.xlabel('Поколение')
111 plt.ylabel('Макс/средняя приспособленность')
112 plt.title('Зависимость максимальной и средней приспособленности от поколения')
113 plt.show()
114
```

Задание к лабораторной работе.

1. Разобраться в решении задачи ONEMAX методом генетического алгоритма.
2. Проанализировать влияние констант генетического алгоритма на решение.